

SEO PACKAGE

1. SEO Title

Why `NULL = NULL` Never Returns TRUE in SQL (And How to Handle Missing Data Correctly)

2. SEO Subtitle

Understand three-valued logic, the `IS NULL` predicate, and `COALESCE` so missing values stop silently breaking your queries.

3. Featured Quote

NULL isn't zero and it isn't empty — it's the database honestly admitting, "I don't know."
The moment you treat "unknown" like a value, your `WHERE` clause starts lying to you.

4. Meta Description

Learn why `NULL = NULL` returns UNKNOWN in SQL, how three-valued logic breaks WHERE clauses, and how to use IS NULL, IS NOT NULL, and COALESCE correctly.

5. URL Slug

`sql-null-equals-null-is-null-coalesce`

6. Keywords

SQL NULL, NULL = NULL, IS NULL, IS NOT NULL, three-valued logic, COALESCE SQL, SQL missing data, WHERE clause NULL, SQL unknown value, handling NULL in SQL, SQL for beginners, database null handling, SQL query filtering, SQL arithmetic NULL, SQL interview questions, learn SQL, relational databases, data cleaning SQL, SQL predicates, SQL operators

7. Tags

SQL, Database, NULL, Data Analytics, Coding Interview, Backend, Data Engineering, SQL Basics, Learn To Code, Relational Databases, Query Optimization, Data Cleaning, Software Engineering, Developer Tutorial

WHY `NULL = NULL` NEVER RETURNS TRUE IN SQL

8. Introduction

Almost every developer meets `NULL` on their first day writing SQL, and almost every developer misunderstands it for a while after that. You write what looks like a perfectly reasonable query, run it, and get back an empty result set — no error, no warning, just silence. Nothing feels worse than a query that is *confidently wrong*.

`NULL` is worth mastering because it sits at the intersection of correctness and data quality.

Interviewers love it precisely because it separates people who memorized syntax from people who understand how the relational model actually reasons about data. When someone asks you, "What does `WHERE phone = NULL` return?" they're really asking, "Do you understand three-valued logic?"

Getting `NULL` right matters far beyond interviews. Real production databases are full of missing values — optional form fields, sensors that failed to report, joins that didn't match. If you treat those gaps as zeros or empty strings, your reports quietly drift away from the truth. This lesson gives you the small, exact vocabulary you need so missing data stops surprising you.

9. Problem Overview

In SQL, `NULL` is a marker that means **the value is unknown or missing**. That is the entire idea, but the consequences are large.

The trap is that `NULL` is not a normal value, so it does not behave like one. It is:

- **not zero** — zero is a number you *know*
- **not an empty string** `''` — an empty string is text you *know* is blank
- **not FALSE** — FALSE is a definite answer

`NULL` is the absence of knowledge. Because you can't meaningfully compare "unknown" to anything using `=`, SQL refuses to give you a clean TRUE or FALSE. Instead it introduces a third truth value:

UNKNOWN. Your job is to learn the two special predicates (`IS NULL` , `IS NOT NULL`) that test for missing data correctly, and the function (`COALESCE`) that substitutes a usable value when one is missing.

Two quick definitions before we go further, because we'll lean on them:

- A **predicate** is a test that answers true or false — for example, "is this row rented out?"
- An **operator** is a symbol, like `=` , that compares two things.

The core problem is this: the `=` operator cannot correctly test for `NULL` , and beginners keep trying to make it.

10. Example

Imagine a `customers` table where some people never entered a phone number:

	id	name	phone	balance
1	Aisha	555-0100	120	
2	Ben	NULL	0	
3	Carmen	555-0199	NULL	
4	Dev	NULL	45	

Look closely at rows 2 and 3, because they encode *three different meanings*:

- **Ben's balance is 0** — we know for certain he owes nothing.
- **Ben's phone is NULL** — we never captured a number; it's unknown.
- **Carmen's balance is NULL** — we don't actually know what she owes; the field was never filled in.

If you collapse all of these into "0" or "blank," you lose information. A `0` balance is a fact. A `NULL` balance is a question mark. Reporting them the same way is how a finance dashboard ends up subtly wrong.

Now the classic mistake:

```
SELECT name FROM customers WHERE phone = NULL;
```

You'd expect Ben and Dev. You get **zero rows** — even though two rows genuinely have a `NULL` phone. Let's understand why.

11. Intuition

Here's the mental shift that makes everything click.

In everyday life, a yes/no question has two answers. SQL adds a third: **UNKNOWN**. And it adds it for a very honest reason.

Ask yourself: *"Is an unknown phone number equal to an unknown phone number?"* You can't answer that. If you don't know either value, you can't claim they're equal, and you can't claim they're different. The only truthful answer is "I don't know" — UNKNOWN.

So any comparison involving `NULL` — `phone = NULL`, `phone <> NULL`, even `NULL = NULL` — evaluates to UNKNOWN, not TRUE and not FALSE.

Now layer on the second fact, and the mystery dissolves completely:

A `WHERE` clause keeps a row only when its condition is TRUE. UNKNOWN is not TRUE, so the row is dropped.

That single rule explains nearly every `NULL` surprise you'll ever hit. `WHERE phone = NULL` produces UNKNOWN for *every* row — including the ones that truly are `NULL` — so every row is filtered out. The rows you were hunting for slip past precisely because you asked the question the wrong way.

An experienced engineer internalizes this as a reflex: *the second I compare against `NULL` with an operator, I've already lost*. That reflex is what pushes you toward the correct tools.

12. Brute Force Solution

Idea: The naive approach is to reach for the equality operator, the same tool you use for every other comparison, and write `= NULL` or `<> NULL`.

Algorithm: 1. Filter with `WHERE phone = NULL` to find missing phones. 2. Filter with `WHERE phone <> NULL` to find present phones.

```
-- Both of these are WRONG
SELECT name FROM customers WHERE phone = NULL;    -- returns nothing
SELECT name FROM customers WHERE phone <> NULL;    -- also returns nothing
```

Advantages: - It's the intuitive first guess — it reuses syntax you already know. - There's genuinely nothing else to learn to *write* it.

Disadvantages: - It is silently, dangerously wrong. No error is raised. - Both queries return an empty set, so you might wrongly conclude "no missing data exists." - It fails identically across every SQL dialect, so switching databases won't save you.

Time complexity: $O(n)$ — a full scan. **Space complexity:** $O(1)$.

The complexity looks fine. The *correctness* is broken, which is the only thing that matters.

13. Optimal Solution

The fix is to stop using operators and start using **predicates designed for missing data**.

IS NULL tests whether a value is missing and returns a real TRUE or FALSE — never UNKNOWN.

IS NOT NULL is the exact inverse: TRUE when a value is present.

Here's how the three truth values behave, so you can see why the predicates escape the trap:

Expression	Result	Kept by WHERE?
phone = NULL	UNKNOWN	✗ No
phone <> NULL	UNKNOWN	✗ No
phone IS NULL	TRUE/FALSE	✓ When TRUE
phone IS NOT NULL	TRUE/FALSE	✓ When TRUE

There's a second front where **NULL** causes damage: **arithmetic**. Any calculation touching **NULL** produces **NULL**, because unknown plus anything is still unknown:

```
100 + NULL -> NULL
balance * 2 -> NULL (when balance is NULL)
SUM(balance) -> ignores NULLs, but a single NULL in a row-level formula poisons that row
```

One missing value can wipe out an entire computed column and quietly deflate a total.

The rescue is **COALESCE**, which reads like: *"use this; if it's missing, use that instead."* It walks a list of expressions left to right and returns the first one that isn't **NULL**:

```
COALESCE(phone, 'No phone on file')
COALESCE(balance, 0)
```

Now you can filter with **IS NULL** and patch with **COALESCE**. They are not competitors — one *finds* missing data, the other *fills* it.

14. Python Code

NULL -style logic isn't unique to SQL — the same "unknown propagates" behavior shows up whenever you model missing data. Here's a small, clean Python simulation that mirrors SQL's three-valued logic and the IS NULL / COALESCE tools, so you can *feel* the semantics in code you can run.

```

"""Simulate SQL's three-valued NULL logic in Python."""

from typing import Any, Optional

UNKNOWN = "UNKNOWN" # SQL's third truth value

def sql_equals(a: Any, b: Any) -> str:
    """Mimic SQL '=': any comparison touching NULL yields UNKNOWN."""
    if a is None or b is None:
        return UNKNOWN
    return str(a == b).upper() # 'TRUE' or 'FALSE'

def is_null(value: Any) -> bool:
    """Mimic SQL 'IS NULL' – always a real True/False, never UNKNOWN."""
    return value is None

def coalesce(*values: Any) -> Optional[Any]:
    """Return the first non-NULL value, like SQL COALESCE."""
    for value in values:
        if value is not None:
            return value
    return None

def where(rows: list[dict], predicate) -> list[dict]:
    """A WHERE clause keeps a row only when the predicate is exactly True."""
    return [row for row in rows if predicate(row) is True]

if __name__ == "__main__":
    customers = [
        {"name": "Aisha", "phone": "555-0100", "balance": 120},
        {"name": "Ben", "phone": None, "balance": 0},
        {"name": "Carmen", "phone": "555-0199", "balance": None},
        {"name": "Dev", "phone": None, "balance": 45},
    ]

    # WRONG: '= NULL' evaluates to UNKNOWN for every row -> nothing matches.
    wrong = where(customers, lambda r: sql_equals(r["phone"], None) == "TRUE")
    assert wrong == [], "phone = NULL must silently return no rows"

```

```
# RIGHT: IS NULL finds the truly missing phones.
missing = where(customers, lambda r: is_null(r["phone"]))
assert [r["name"] for r in missing] == ["Ben", "Dev"]

# COALESCE patches missing balances with 0 for a safe total.
total = sum(coalesce(r["balance"], 0) for r in customers)
assert total == 165 # 120 + 0 + 0 + 45

print("Missing phones :", [r["name"] for r in missing])
print("Safe total      :", total)
```

Running it prints the two customers with missing phones and a correct total of `165` , and the `assert` s fail loudly if any of the `NULL` rules are broken.

15. Code Walkthrough

- `sql_equals` is the heart of the lesson. The very first thing it checks is whether either operand is `None` (Python's stand-in for `NULL`). If so, it returns `"UNKNOWN"` — never `TRUE`, never `FALSE` — exactly like SQL's `=` operator.
- `is_null` always returns a genuine `bool` . This is why `IS NULL` works where `=` fails: it escapes the three-valued trap entirely.
- `coalesce` loops through its arguments and returns the first that isn't `None` . If everything is missing, it returns `None` , matching SQL's behavior when every argument is `NULL` .
- `where` encodes the rule that makes `NULL` confusing: `if predicate(row) is True` . The strict `is True` check means an `"UNKNOWN"` result (or anything that isn't literally `True`) causes the row to be dropped — mirroring how SQL's `WHERE` discards non-`TRUE` rows.
- The `__main__` **block** is a self-check. The `assert wrong == []` line proves the classic bug: comparing with `NULL` silently returns nothing. The `assert missing == ["Ben", "Dev"]` line proves the fix.

16. Dry Run

Let's trace the "find missing phones" query against our four customers using `IS NULL` :

Row	phone	is_null(phone)	Kept?
Aisha	555-0100	FALSE	✗
Ben	None	TRUE	✓
Carmen	555-0199	FALSE	✗
Dev	None	TRUE	✓

Result: **Ben and Dev** — correct.

Now the same rows with the buggy `phone = NULL`:

Row	<code>sql_equals(phone, None)</code>	<code>== "TRUE" ? Kept?</code>
Aisha	UNKNOWN	No 
Ben	UNKNOWN	No 
Carmen	UNKNOWN	No 
Dev	UNKNOWN	No 

Result: **nothing**. Every comparison against `NULL` collapsed to UNKNOWN, so `WHERE` swept the whole table away — including the rows you wanted.

17. Complexity Analysis

For a straightforward filtered scan over `n` rows:

- **Time: $O(n)$.** Whether you use `= NULL` (wrong) or `IS NULL` (right), the database evaluates the predicate once per row. The predicate is $O(1)$, so the scan dominates. An index on the column can reduce this in practice, but the worst case remains a full scan.
- **Space: $O(1)$** for the evaluation itself (plus $O(k)$ to hold the k matching rows you return).

The key insight: **correctness and performance are independent here**. The wrong query isn't slow — it's fast and empty. That's what makes it dangerous.

18. Alternative Solutions

A few reasonable alternatives to `COALESCE`, and how they compare:

- `IFNULL(x, y)` / `ISNULL(x, y)` — a two-argument shortcut in MySQL and SQL Server respectively. Fine for a single fallback, but not portable and limited to two values. Prefer `COALESCE` for cross-dialect code.
- `NULLIF(a, b)` — the reverse tool: it *produces* `NULL` when `a = b`. Handy for turning a sentinel like `-1` or `''` back into a proper `NULL`.
- `CASE WHEN x IS NULL THEN y ELSE x END` — the fully explicit form. `COALESCE` is essentially syntactic sugar for this; reach for `CASE` only when the fallback logic is more complex than "first non-null wins."

For **filtering**, there is no real alternative to `IS NULL` / `IS NOT NULL`. They are the only correct predicates for the job.

19. Edge Cases

- **Empty table:** `IS NULL` correctly returns zero rows — no missing data because there's no data.
- **All values NULL:** `COALESCE(col, default)` returns the default for every row; `SUM(col)` returns `NULL` (or `0` only if you `COALESCE` it).
- **NULL inside IN lists:** `x IN (1, 2, NULL)` can behave unexpectedly — a non-match becomes UNKNOWN rather than FALSE. Watch `NOT IN` especially: `x NOT IN (1, NULL)` returns *no rows*.
- **NULL in JOIN keys:** `NULL` never equals `NULL`, so rows with `NULL` join keys won't match — a common source of "lost" rows.
- **NULL with aggregates:** `COUNT(col)` skips `NULL`s, but `COUNT(*)` counts every row. Choose deliberately.
- **NULL in ORDER BY :** sort placement (`NULLS FIRST` / `NULLS LAST`) varies by database.

20. Common Mistakes

1. **Using `= NULL` or `<> NULL`.** Both yield UNKNOWN and silently return nothing. Always use `IS NULL` / `IS NOT NULL`.
2. **Treating `NULL` as `0` or `''`.** These are known values; `NULL` is the absence of one. Conflating them corrupts totals and logic.
3. **Forgetting arithmetic propagation.** `price + NULL` is `NULL`, and one `NULL` can void an entire computed column. Wrap risky operands in `COALESCE`.
4. **NOT IN with a NULL in the list.** This can eliminate all rows unexpectedly. Filter `NULL`s out of the subquery or use `NOT EXISTS`.
5. **Assuming `COUNT(column)` counts every row.** It ignores `NULL`s; use `COUNT(*)` when you want the true row count.
6. **Expecting `NULL` join keys to match.** They never do — `NULL = NULL` is UNKNOWN even inside a `JOIN`.

21. Interview Questions

- What does `SELECT * FROM t WHERE col = NULL` return, and why?
- Explain three-valued logic. What is the third value and where does it come from?
- What's the difference between `NULL`, `0`, and `''`?
- How does `COUNT(col)` differ from `COUNT(*)` when `NULL`s are present?
- Why can `NOT IN` with a `NULL` in the list return zero rows?

- When would you choose `COALESCE` over `CASE` ? Over `IFNULL` ?
- How does `NULL` affect an `INNER JOIN` vs. a `LEFT JOIN` ?

22. Similar Problems

- **LeetCode 175 – Combine Two Tables:** a `LEFT JOIN` where unmatched rows produce `NULL` s you must handle correctly.
- **LeetCode 577 – Employee Bonus:** explicitly requires selecting employees whose bonus `IS NULL` or is below a threshold — the classic `NULL` -in-filter trap.
- **LeetCode 1148 – Article Views I** and **595 – Big Countries:** filtering practice where careless `NULL` handling drops valid rows.
- **LeetCode 619 – Biggest Single Number:** leans on `NULL` as the "no result" answer via aggregation.

They're related because each one punishes the assumption that `NULL` behaves like an ordinary value in a `WHERE` , `JOIN` , or aggregate.

23. Key Takeaways

- `NULL` means **unknown**, not zero and not empty.
- Any comparison with `NULL` returns **UNKNOWN**, and `WHERE` keeps only `TRUE` rows — so `= NULL` silently returns nothing.
- Use `IS NULL` and `IS NOT NULL` to test for missing data. They're your only correct tools.
- `NULL` **propagates through arithmetic** — one missing value voids the whole calculation.
- Use `COALESCE` to substitute a usable default. `IS NULL` filters; `COALESCE` fixes. Keep both.

24. Watch the Video

Reading the rules is one thing — *seeing* a query return an empty result set and then watching `IS NULL` rescue it makes the idea stick. Kai walks through every one of these traps live, with real queries, in Lesson 6 of the SQL series. Watch it here: {LINK}

25. About the Series

This is part of **Fun with Learning Technology**, a beginner-friendly series that takes one concept at a time and teaches the *intuition* first — never syntax for its own sake. Each lesson builds on the last, so

by the end you're not memorizing SQL, you're reasoning in it. If you're learning databases, algorithms, or backend engineering, follow along from Lesson 1.

26. Call To Action

If this cleared up `NULL` for you:

- 👍 **Like** the video on YouTube — it genuinely helps the channel grow.
 - 🔔 **Subscribe** so you catch the next lesson.
 - 📖 **Subscribe on Substack** for the written companion to every video.
 - 💬 **Comment** with the `NULL` bug that once caught you — we've all got one.
 - 📢 **Share** this with a teammate who's still writing `= NULL`.
-

SOCIAL MEDIA

LinkedIn Post

Early in my career I stored a customer's balance as `NULL`, then spent an afternoon wondering why my totals looked wrong.

Here's the lesson that fixed it — and it trips up seniors as often as beginners.

In SQL, `NULL` doesn't mean zero. It doesn't mean an empty string. It means *unknown*. The database is honestly telling you: "I never found out what belongs here."

That distinction matters more than it looks, because SQL uses three-valued logic. Normal life is yes or no. SQL adds a third answer: `UNKNOWN`. And the instant you compare anything to `NULL` — even `NULL = NULL` — you get `UNKNOWN`, not `TRUE` and not `FALSE`.

Now combine that with one more rule: a `WHERE` clause keeps a row only when the condition is `TRUE`.

So this query:

```
WHERE phone = NULL
```

returns *nothing* — even the rows that genuinely have a missing phone. No error. No warning. Just a silently empty result set. That's the dangerous part: the query isn't slow, it's confidently wrong.

The fix is two purpose-built predicates:

- IS NULL — find the missing values
- IS NOT NULL — find the present ones

And when NULL sneaks into arithmetic ($100 + \text{NULL} = \text{NULL}$, which can void an entire total), COALESCE steps in to supply a sensible default.

The mental model that makes it all click: IS NULL is for *filtering*, COALESCE is for *fixing*. They're different tools — keep both.

Master this and missing data stops surprising you.

What's the NULL bug that once cost you an afternoon?

SQL #Database #DataAnalytics

#SoftwareEngineering #LearnToCode

Twitter/X Thread

1/ `NULL = NULL` never returns TRUE in SQL.

This one line of trivia explains 90% of the "why is my query empty?" bugs beginners AND seniors hit.

Let me fix it for you in 9 tweets. 🗣️

2/ First: what is NULL?

It is NOT zero. It is NOT an empty string ". It means UNKNOWN — the database saying "I never found out what belongs here."

Zero is a value you know. NULL is the absence of one. Three different animals.

3/ Normal logic has two answers: TRUE and FALSE.

SQL adds a third: UNKNOWN.

And here's the trap — ANY comparison with NULL returns UNKNOWN. Not true, not false. Just... unknown.

4/ Now the rule that ruins your day:

A WHERE clause keeps a row ONLY when the condition is TRUE.

UNKNOWN $\neq$ TRUE. So UNKNOWN rows quietly vanish.

5/ Put those together:

```
WHERE phone = NULL
```

returns NOTHING — even the rows that truly have a missing phone.

No error. No warning. Just a silently empty result. That's the classic beginner trap.

6/ The fix is a special predicate built for this exact job:

```
WHERE phone IS NULL
```

IS NULL returns a REAL true/false instead of UNKNOWN. Now the missing rows actually show up.

7/ Need the opposite — only rows that HAVE a phone?

```
WHERE phone IS NOT NULL
```

And never reach for `<> NULL`. It falls into the same UNKNOWN hole. IS NULL / IS NOT NULL are your only two tools.

8/ NULL also poisons math:

```
100 + NULL -> NULL
```

One missing value can wipe out an entire total before you notice.

Rescue: COALESCE(x, o) → "use x, but if it's missing, use o instead."

9/ The mental model:

- IS NULL → for FILTERING missing rows
- COALESCE → for FIXING missing values

Different jobs. Keep both.

Master NULL and missing data stops surprising you. Full lesson 🙌 [{LINK}](#)

Facebook Post

Ever written a SQL query that returned zero rows — no error, just... nothing — and had no idea why?



There's a good chance NULL was the culprit.

Here's the quick version: in SQL, NULL doesn't mean zero and it doesn't mean blank. It means *unknown*. And because you can't compare "unknown" to anything, writing `WHERE phone = NULL` returns NOTHING — even the rows that really do have a missing phone!

The fix is simple once you know it: use `IS NULL` and `IS NOT NULL` to find missing data, and `COALESCE` to fill in a sensible default. That's the whole trick.

I broke it all down step by step in Lesson 6 of the SQL series — real queries, real "aha" moments. If you're learning SQL (or you've ever been burned by NULL), this one's for you. 📌 {LINK}

Reddit Summary

Why `NULL = NULL` never returns TRUE in SQL — a clean mental model

A lot of "my query returns no rows and I don't know why" questions come down to one thing: how NULL behaves. Sharing the model that finally made it click for me.

- **NULL means *unknown*.** Not zero, not `' '`. Those are values you *know*; NULL is the absence of one.
- **SQL uses three-valued logic:** TRUE, FALSE, and UNKNOWN. Any comparison involving NULL — including `NULL = NULL` — evaluates to UNKNOWN.
- **WHERE keeps a row only when the condition is TRUE.** So `WHERE col = NULL` returns nothing, because every comparison is UNKNOWN, not TRUE — even for rows that genuinely are NULL.
- **Correct predicates:** `IS NULL` and `IS NOT NULL`. These return a real true/false and escape the trap. `<> NULL` does not — same UNKNOWN hole.
- **NULL propagates through arithmetic:** `100 + NULL = NULL`, so one missing value can void a total.
- **COALESCE(col, default)** returns the first non-NULL argument — good for defaults and for stopping NULL from poisoning sums.

Rule of thumb: `IS NULL` is for *filtering* (find/exclude missing rows), `COALESCE` is for *fixing* (swap missing for usable). Different tools, keep both.

Gotchas worth knowing: `NOT IN (... , NULL)` can return zero rows; `COUNT(col)` skips NULLs while `COUNT(*)` doesn't; NULL join keys never match.

Curious how others explain three-valued logic to newer devs — what analogy worked for you?

GITHUB README

SQL: Why `NULL = NULL` Never Returns TRUE

Lesson 6 of the SQL series – understanding NULL, three-valued logic, `IS NULL` / `IS NOT NULL`, and `COALESCE`.

Problem

In SQL, `NULL` marks a **missing or unknown** value. It is not `0` and not `''`. Because you can't meaningfully compare "unknown" to anything, comparisons using `=` or `<>` against `NULL` return **UNKNOWN** – never TRUE. Since a `WHERE` clause keeps only rows where the condition is TRUE, queries like `WHERE col = NULL` silently return **no rows** – even rows that truly are NULL.

Intuition

- Everyday logic: TRUE / FALSE.
- SQL logic: TRUE / FALSE / **UNKNOWN**.
- Any comparison touching `NULL` → UNKNOWN.
- `WHERE` drops non-TRUE rows → NULL rows vanish under `=`.

The takeaway: the `=` operator can't test for NULL. You need dedicated predicates.

Approach

Goal	Use
Find missing values	`col IS NULL`
Find present values	`col IS NOT NULL`
Replace a missing value	`COALESCE(col, default)`

`IS NULL` / `IS NOT NULL` return real TRUE/FALSE and escape three-valued logic. `COALESCE` returns the first non-NULL argument, protecting arithmetic and reports from NULL propagation (e.g. `100 + NULL = NULL`).

SQL Examples

```
```sql
-- WRONG: returns nothing, no error
SELECT name FROM customers WHERE phone = NULL;

-- RIGHT: find customers with no phone
SELECT name FROM customers WHERE phone IS NULL;

-- RIGHT: find customers who have a phone
SELECT name FROM customers WHERE phone IS NOT NULL;

-- Protect a total from NULL poisoning
SELECT SUM(COALESCE(balance, 0)) AS total_balance FROM customers;
```

## Python Simulation

```
UNKNOWN = "UNKNOWN"

def sql_equals(a, b):
 """Any comparison with NULL yields UNKNOWN."""
 if a is None or b is None:
 return UNKNOWN
 return str(a == b).upper()

def is_null(value):
 """IS NULL — always a real True/False."""
 return value is None

def coalesce(*values):
 """First non-NULL value, like SQL COALESCE."""
 for v in values:
 if v is not None:
 return v
 return None

def where(rows, predicate):
 """WHERE keeps a row only when the predicate is exactly True."""
 return [r for r in rows if predicate(r) is True]

if __name__ == "__main__":
 customers = [
 {"name": "Aisha", "phone": "555-0100", "balance": 120},
 {"name": "Ben", "phone": None, "balance": 0},
 {"name": "Carmen", "phone": "555-0199", "balance": None},
 {"name": "Dev", "phone": None, "balance": 45},
]
 assert where(customers, lambda r: sql_equals(r["phone"], None) == "TRUE") == []
 assert [r["name"] for r in where(customers, lambda r: is_null(r["phone"]))] == ["Ben", "Dev"]
 assert sum(coalesce(r["balance"], 0) for r in customers) == 165
 print("OK")
```

## Complexity

- **Time:**  $O(n)$  — one predicate evaluation per row (a full scan; an index can help).
- **Space:**  $O(1)$  for evaluation,  $O(k)$  for the  $k$  returned rows.
- Note: the buggy `= NULL` query is just as fast — it's *correctness* that fails.

## Common Mistakes



1. `= NULL` / `<> NULL` → always UNKNOWN, returns nothing.
2. Treating NULL as `0` or `''`.
3. Forgetting NULL propagates through arithmetic.
4. `NOT IN (... , NULL)` returning zero rows.
5. Assuming `COUNT(col)` counts NULLs (it doesn't; `COUNT(*)` does).

## Video

---

 Watch Lesson 6: {LINK}

## Article

---

 Full written walkthrough: {LINK}

## Series

---

Part of **Fun with Learning Technology** — beginner-friendly lessons in SQL, algorithms, and software engineering, taught intuition-first. ``

## Quick Review

---

### SQL: when you must match rows with missing data, what predicate?

Use `IS NULL` / `IS NOT NULL`. NULL means unknown, so it never equals anything — including itself.

### Why does ``column = NULL`` never return rows?

NULL is unknown, so any comparison yields UNKNOWN (not TRUE). WHERE keeps only TRUE rows, so nothing matches.

### SQL three-valued logic: what results can a comparison give?

TRUE, FALSE, or UNKNOWN. Comparing anything to NULL gives UNKNOWN, and WHERE filters out non-TRUE rows.

**Trickiest bug: what does NULL do inside arithmetic like `price + tax`?**

It poisons the result — any math with NULL returns NULL, silently dropping or blanking values in aggregates and expressions.

**IS NULL vs COALESCE — when to use each?**

IS NULL filters/detects missing rows in WHERE. COALESCE substitutes a fallback value, e.g. COALESCE(col, 0), so math and output stay clean.

**Interview: how do you count rows where a column has no value?**

SELECT COUNT(\*) ... WHERE col IS NULL. COUNT(col) alone ignores NULLs, so it wouldn't count them.

---

sql Lesson 6: NULL Values and IS NULL (NULL represents the absence of data, distinguishing an unknown value from a zero or an empty string. Understanding how NULL interacts with logical operations is vital for preventing data loss or incorrect query filtering results. You must use specific predicates rather than equality operators to identify or exclude these missing values in your database.) — free Python cheat sheet